



Eduvos
Your Education. Your Future.

Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus:	Tygervalley
Faculty:	Information Technology
Module Code:	ITCMA2-22
Group:	National
Lecturer's Name:	Dr Oluwaseyi Babalola
Student Full Name:	Joshua Du Plooy
Student Number:	CON-828169-S6R1

Indicate	Yes	No
Plagiarism report attached	X	

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature 	Date 14 MARCH 2025
---	------------------------------

Lecturer's Comments:

--

Marks Awarded:	%
-----------------------	---

Signature	Date
------------------	-------------

DIGITAL ELECTRONICS AND MICROCONTROLLERS

AN

INDIVIDUAL PROJECT

*Submitted for Partial fulfillment of the Requirement
For the award of the degree of*

BACHELOR OF SCIENCE

IN

INFORMATION TECHNOLOGY (ROBOTICS)

Submitted by

JOSHUA DU PLOOY

(Registration no. CON-828169-S6R1)

Under the guidance of

Dr Oluwaseyi Babalola

Lecturer



DEPARTMENT OF INFORMATION TECHNOLOGY

Contents

1	Question 1	1
1.1	Question 1.1	1
1.2	Question 1.2	4
2	Question 2	6
2.1	Question 2.1	6
2.1.1	Question 2.1.2	7
2.1.2	Question 2.1.3	7
2.1.3	Question 2.1.4	8
2.2	Question 2.1	9
2.2.1	Question 2.2.1	10
2.2.2	Question 2.2.2	11
2.2.3	Question 2.1.4	12
3	Question 3	17
3.1	Question 3.1	17
3.2	Question 3.2	17
3.3	Question 3.3	23
3.4	Question 3.4	23
3.5	Question 3.5	24
3.6	Question 3.6	28
3.7	Question 3.7	28
4	Question 4	29
4.1	Question 4.1	29
4.1.1	Issue 1	29
4.1.2	Issue 2	30
4.2	4.2	30
	Bibliography	32

Chapter 1: Question 1

Alex is an aspiring inventor keen on creating a basic locking mechanism for his room using an Arduino Uno board. He envisions a system where a push button serves as the key to open the door. After some additional thought, Alex realized that his initial solution is very intuitive, and everyone could have access to his room. He decides to improve on that and use a combination of many push buttons and store the code in the memory of the Arduino board. A combination of those push buttons will be used to unlock the door. Alex wants you to help achieve that project using the Arduino Board and the components you have available.// From the Arduino board picture below, identify all Inputs and Outputs using the formats provided in the table below:

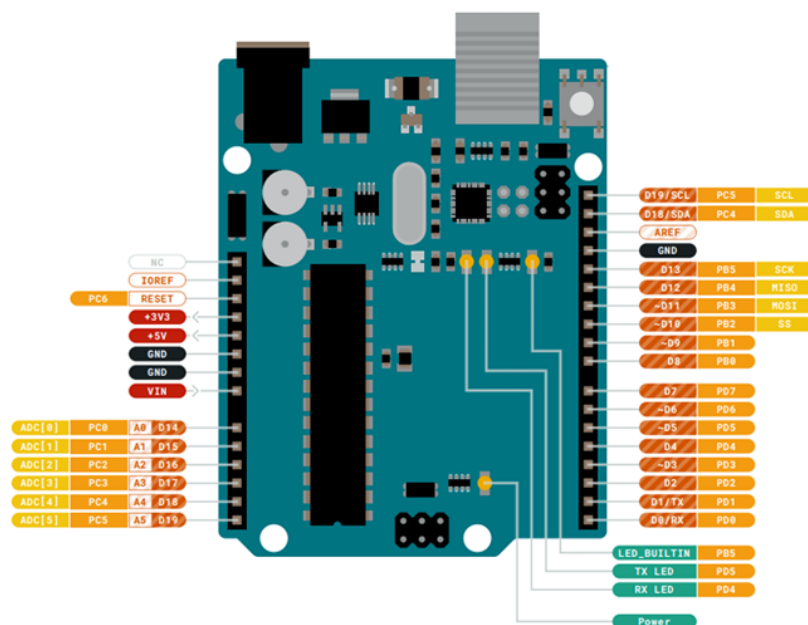


Figure 1.1: Arduino Pinout

1.1 Question 1.1

From the Arduino board picture below, identify all Inputs and Outputs:

Pin	Function	Type	Description
1	NC	NC	Not Connected
2	IOREF	IOREF	The reference level for digital logic, usually used for shields - typically connected to 5V on arduino board
3	PC6 RESET	AtMega Reset	Pin 6 Port C on the AtMega Processor RESET Pin resets the arduino board when pulled LOW
4	+3V3	Power	3.3V power pin supplied by the arduino board when powered
5	+5V	Power	5V power pin supplied by the arduino board when powered
6	GND	Power	common Ground pin for the arduino board
7	GND	Power	common Ground pin for the arduino board
8	VIN	Power	Voltage Input power pin that can be used to power the arduino board with a minimum of 6V and a maximum of 20V
9	ADC[0] PC0 A0 D14	ADC AtMega Analog Digital/GPIO	Analog to Digital Converter Channel 0 Pin 0 Port C on the AtMega Processor Analog input Pin 0 Digital General Purpose input output pin 14
10	ADC[1] PC1 A1 D15	ADC AtMega Analog Digital/GPIO	Analog to Digital Converter Channel 1 Pin 1 Port C on the AtMega Processor Analog input Pin 1 Digital General Purpose input output pin 15
11	ADC[2] PC2 A2 D16	ADC AtMega Analog Digital/GPIO	Analog to Digital Converter Channel 2 Pin 2 Port C on the AtMega Processor Analog input Pin 2 Digital General Purpose input output pin 16
12	ADC[3] PC3 A3 D17	ADC AtMega Analog Digital/GPIO	Analog to Digital Converter Channel 3 Pin 3 Port C on the AtMega Processor Analog input Pin 3 Digital General Purpose input output pin 17
13	ADC[4] PC4 A4 D18 SDA	ADC AtMega Analog Digital/GPIO Digital/I2C	Analog to Digital Converter Channel 4 Pin 4 Port C on the AtMega Processor Analog input Pin 4 Digital General Purpose input output pin 18 I2C Data line
14	ADC[5] PC5 A5 D19 SCL	ADC AtMega Analog Digital/GPIO Digital/I2C	Analog to Digital Converter Channel 5 Pin 5 Port C on the AtMega Processor Analog input Pin 5 Digital General Purpose input output pin 19 I2C Clock line

Table 1.1: Analog

Pin	Function	Type	Description
1	D0 RX PD0	Digital/GPIO UART AtMega	Digital General Purpose input output pin 0 Digital Receive pin Pin 0 Port D on the AtMega Processor
2	D1 TX PD1	Digital/GPIO UART AtMega	Digital General Purpose input output pin1 Digital Transmit pin Pin 1 Port D on the AtMega Processor
3	D2 PD2	Digital/GPIO AtMega	Digital General Purpose input output pin 2 Pin 2 Port D on the AtMega Processor
4	D3 PD3	Digital/GPIO AtMega	Digital General Purpose input output pin 3 Pin 3 Port D on the AtMega Processor
5	D4 PD4	Digital/GPIO AtMega	Digital General Purpose input output pin 4 Pin 4 Port D on the AtMega Processor
6	D5 PD5	Digital/GPIO AtMega	Digital General Purpose input output pin 5 Pin 5 Port D on the AtMega Processor
7	D6 PD6	Digital/GPIO AtMega	Digital General Purpose input output pin 6 Pin 6 Port D on the AtMega Processor
8	D7 PD7	Digital/GPIO AtMega	Digital General Purpose input output pin 7 Pin 7 Port D on the AtMega Processor
9	D8 PB0	Digital/GPIO AtMega	Digital General Purpose input output pin 8 Pin 0 Port B on the AtMega Processor
10	D9 PB1	Digital/GPIO AtMega	Digital General Purpose input output pin 9 Pin 1 Port B on the AtMega Processor
11	D10 PB2 SS	Digital/GPIO AtMega Digital/SPI	Digital General Purpose input output pin 10 Pin 2 Port B on the AtMega Processor SPI Chip Select
12	D11 PB3 MOSI	Digital/GPIO AtMega Digital/SPI	Digital General Purpose input output pin 11 Pin 3 Port B on the AtMega Processor SPI1 Main Out Secondary In
13	D12 PB4 MISO	Digital/GPIO AtMega Digital/SPI	Digital General Purpose input output pin 12 Pin 4 Port B on the AtMega Processor SPI Main In Secondary Out
14	D13 PB5 SCK	Digital/GPIO AtMega Digital/SPI	Digital General Purpose input output pin 13 Pin 5 Port B on the AtMega Processor SPI Serial Clock Output
15	GND	Power	common Ground pin for the arduino board
16	AREF	Digital	the reference level for Analog signals, usually used for shields
17	ADC[4] PC4 A4 D18 SDA	ADC AtMega Analog Digital/GPIO Digital/I2C	Analog to Digital Converter Channel 4 Pin 4 Port C on the AtMega Processor Analog input Pin 4 General Purpose input output pin 18 I2C Data line
18	ADC[5] PC5 A5 D19 SCL	ADC AtMega Analog Digital/GPIO Digital/I2C	Analog to Digital Converter Channel 5 Pin 5 Port C on the AtMega Processor Analog input Pin 5 General Purpose input output pin 19 I2C Clock line

Table 1.2: Analog

1.2 Question 1.2

Draw the power tree in the Arduino board and clearly represent all Voltage levels.

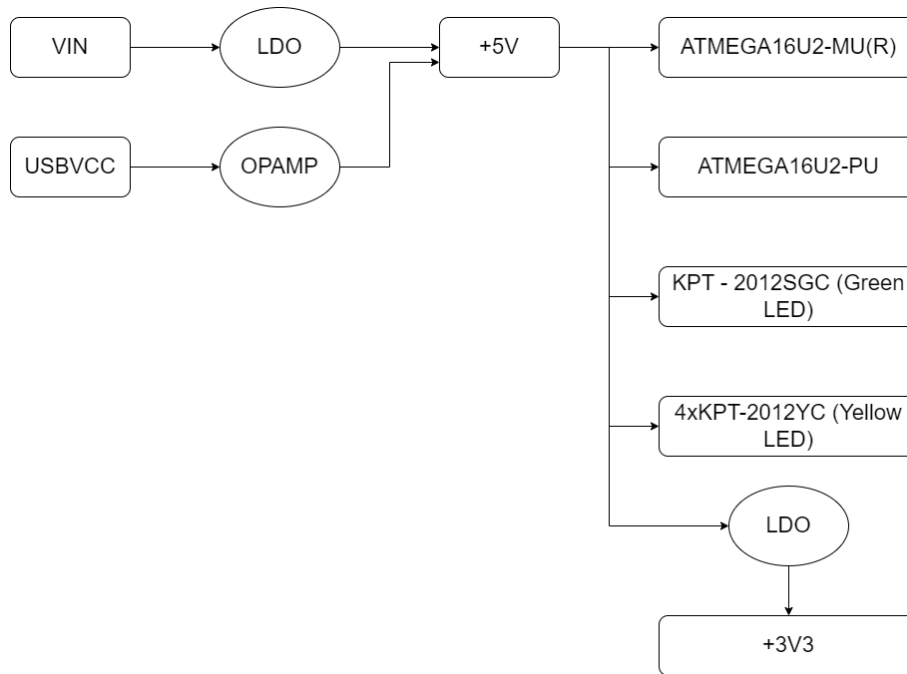


Figure 1.2: Arduino Power Tree Diagram

With Reference to the above Figure:

- The following are physical pins on the Arduino board:
 - VIN
 - USBVCC (the positive power pin inside the USB port)
 - +5V
 - +3V3
- The following are physical components on the Arduino board:
 - ATMEGA16U2-MU(R) (The USB to serial chip that interfaces with the computer - i.e. a Multiplexer) (max Current Rating of 261mA)
 - ATMEGA328P-PU (The main microcontroller on the Arduino board) (max Current Rating of 410mA)
 - KPT-2012SGC (Green LED) (The green LED indicating when the Arduino is ON) (max Current Rating of 5.6mA)
 - 4x KPT-2012YC (Yellow LEDs) (An LED for the board error status, Pin 13, RX and TX) (max Current Rating of 8.7mA)

- The following are physical Components on the arduino board used to regulate voltage:
 - LDO (Low Dropout Regulator, used to regulate DC voltage)
 - OPAMP (Operational Amplifier, used to Amplify, filter, inverse and isolate voltage and current)

Chapter 2: Question 2

2.1 Question 2.1

Basic Alarm System

A student wants to create a basic alarm system for their desk drawer using an Arduino Uno. The system uses a push button as a sensor (e.g., drawer open/close detection). When the drawer is opened (the button is released), an LED should turn on to indicate the breach.

Requirements:

1. Use a pull-down resistor for the button input.
2. List all necessary components.
3. Provide the schematic diagram.
4. Write the Arduino sketch that activates the LED when the drawer is opened.

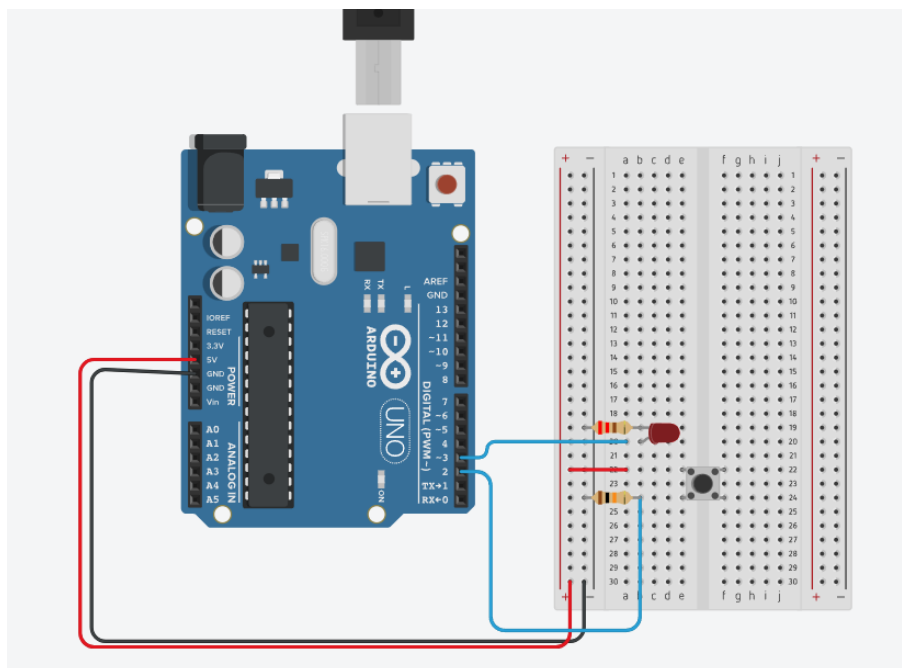


Figure 2.1: Arduino-Breadboard Connection Diagram

2.1.1 Question 2.1.2

Name	Quantity	Component
V5	1	Arduino Uno R3
S1	1	Push button
R5	1	1k Ω Resistor
D1	1	Red LED
R2	1	220 Ω Resistor

Table 2.1: Component List

2.1.2 Question 2.1.3

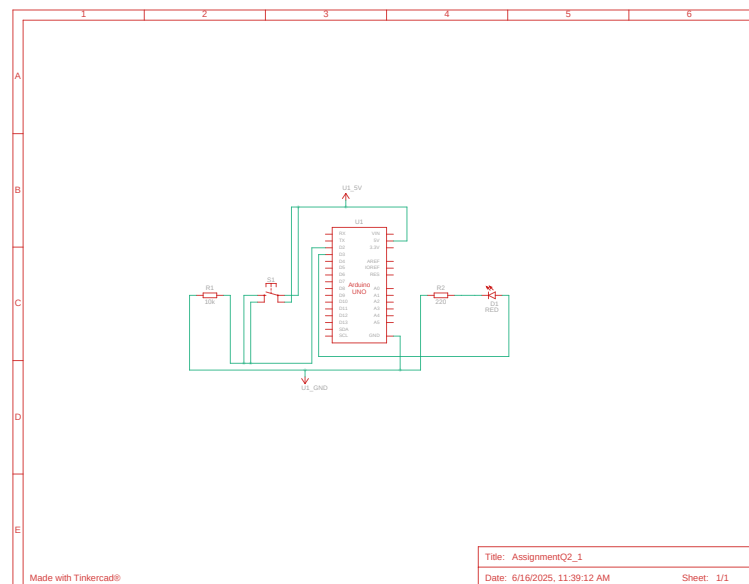


Figure 2.2: Schematic Diagram

2.1.3 Question 2.1.4

```
// C++ code
//
const byte B2=2;
const byte led=3;

byte B2state;
void setup()
{
    pinMode(led, OUTPUT);
    pinMode(B2, INPUT);
    B2state=digitalRead(B2);
    digitalWrite(led,!B2state);
}

void loop()
{
    byte B2cur=digitalRead(B2);
    if (B2state!=B2cur){
        B2state=B2cur;
        digitalWrite(led,!B2state);
    }
}
```

2.2 Question 2.1

Alarm System with Passcode Memory and Buzzer Alert

The student wants to upgrade the alarm system by introducing two push buttons (Button A and Button B) and a 4-digit passcode saved in EEPROM memory. The system should work as follows:

- The LED alarm turns on when the drawer opens.
- The buzzer sounds until the correct code combination (e.g., AABB) is entered.
- The code is stored in EEPROM and checked on every attempt
- A servo motor rotates to unlock the drawer only if the correct combination is entered.

Instructions:

- List all necessary components.
- Provide a complete schematic diagram.
- Write the full Arduino sketch, including EEPROM functions, input handling, and logic processing.

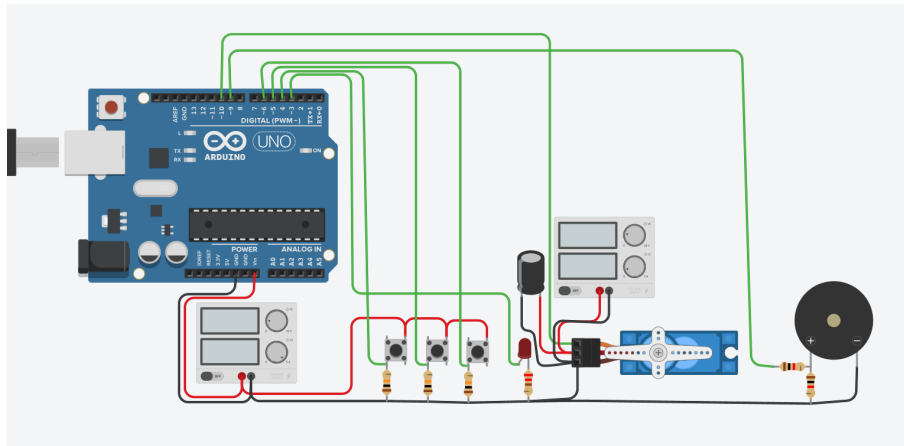


Figure 2.3: Arduino Connection Diagram

2.2.1 Question 2.2.1

Name	Quantity	Component
P1	1	5V Power Supply
U1	1	Arduino Uno R3
S1 S2 S3	3	Pushbutton
R3 R4 R5	3	10kΩ Resistor
D1	1	Red LED
R6	1	220Ω Resistor
PIEZO1	1	Piezo
R1 R2	2	1kΩ Resistor
SERVO1	1	Positional Micro Servo
C1	1	100 uF, 7 V Polarized Capacitor
P2	1	6V , 500mA Power Supply

Table 2.2: Component List

2.2.2 Question 2.2.2

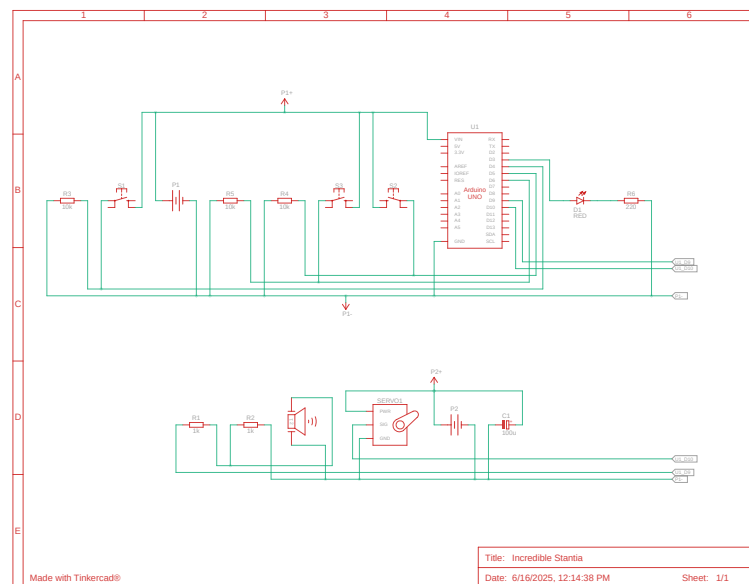


Figure 2.4: Schematic Diagram

2.2.3 Question 2.1.4

```
#include <Servo.h>

#include <EEPROM.h>

// C++ code
//

class Button {
private:
    byte pin;
    bool lastState;
    bool debounceState;
    unsigned long lastDebounce;
    void (*riseEdge)() = nullptr;
    void (*fallEdge)() = nullptr;

public:
    Button(byte p){
        pin=p;
    }
    void begin(){
        pinMode(pin,INPUT);
        lastState = digitalRead(pin);
        debounceState = lastState;
        lastDebounce = millis();
    }
    void setRiseCallback(void (*callback)()) {
        riseEdge = callback;
    }

    void setFallCallback(void (*callback)()) {
        fallEdge = callback;
    }
    void update(){
        bool curState= digitalRead(pin);
        if (lastState!=curState){
            lastDebounce = millis();
```

```

    }
    if ((millis() - lastDebounce) > 50){
        if (curState != debounceState) {
            debounceState = curState;
            if (debounceState && riseEdge){
                riseEdge();
            }
            else if (!debounceState && fallEdge){
                fallEdge();
            }
        }
    }
    lastState=curState;
}

bool getDebounceState(){
    return debounceState;
}
};

```

```

const byte LED=3;

```

```

Button* keypad[2];
const byte buzzer=9;
const byte servo=10;
Button sensor(4);
Button Abutton(5);
Button Bbutton(6);
char passcodeAttempt[5]="0000";
Servo myservo;

```

```

void alarm_on(){
    digitalWrite(LED,1);
    tone(buzzer,800);
    Serial.println("alarm_on");
    myservo.write(90);
}

void alarm_off(){
    myservo.write(90);
}

```

```

digitalWrite(LED,0);
Serial.println("alarm_off");
noTone(buzzer);

}

bool compareCharArrays(const char* a, const char* b) {
    while (*a && *b) {
        if (*a != *b) {
            return false;
        }
        a++;
        b++;
    }
    return *a == *b;
}

void APressed(){
    for (int i=0;i<3;i++){
        passcodeAttempt[i]=passcodeAttempt[i+1];
    }
    passcodeAttempt[3]='A';
    passcodeAttempt[4]='\0';
    Serial.print("Passcode Attempt: ");
    Serial.print(passcodeAttempt);
    Serial.print(" Passcode is: ");

    char passcode[5];
    EEPROM.get(0,passcode);
    Serial.print(passcode);
    Serial.println();
    if (compareCharArrays(passcodeAttempt, passcode)){
        Serial.println("Password Correct, Unlocking...");
        alarm_off();
        myservo.write(0);
    }
}

void BPressed(){
    for (int i=0;i<3;i++){

```

```

    passcodeAttempt[i]=passcodeAttempt[i+1];
}
passcodeAttempt[3]='B';
passcodeAttempt[4]='\0';
Serial.print("Passcode Attempt: ");
Serial.print(passcodeAttempt);
Serial.print("Passcode is: ");

char passcode[5];
EEPROM.get(0,passcode);
Serial.print(passcode);
Serial.println();
if (compareCharArrays(passcodeAttempt, passcode)){
    Serial.println("Password Correct, Unlocking...");
    alarm_off();
    myservo.write(0);
}
}

void setup()
{

    pinMode(LED,OUTPUT);

    sensor.begin();
    sensor.setRiseCallback(alarm_off);
    sensor.setFallCallback(alarm_on);
    if (!sensor.getDebounceState()){
        alarm_on();

    }

    Abutton.begin();
    Bbutton.begin();
    Abutton.setFallCallback(APressed);
    Bbutton.setFallCallback(BPressed);
    Serial.begin(9600);
    EEPROM.put(0,"AABB");//Delete this line before flashing to arduino
    char passcode[5];
    EEPROM.get(0,passcode);

```

```
Serial.print("Passcode is: ");
Serial.print(passcode);
Serial.println();

myservo.attach(10);

}

void loop()
{
  sensor.update();
  Abutton.update();
  Bbutton.update();
}
```

Chapter 3: Question 3

You are tasked with developing a Smart Voting System prototype for student election. The system must use two buttons to represent votes for Candidate A and Candidate B and must only accept a vote if the user presses one of the two buttons — but not both at the same time.

If both buttons are pressed, the system must invalidate the vote and turn on a red LED. When a valid vote is cast, a green LED flashes briefly, and the vote is registered via the Serial Monitor.

The following specifications are critical to the implementation of your design.

- Button A = Vote for Candidate A → pin 2
- Button B = Vote for Candidate B → pin 3
- Green LED = pin 6 (Valid Vote Indicator)
- Red LED = pin 7 (Invalid Vote Indicator)

3.1 Question 3.1

Draw the truth table that summarise the operation of your Smart Voting System.

A	B	Green LED flashes briefly	red LED
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 3.1: Truth Table

3.2 Question 3.2

Draw a diagram and write the Arduino code for a program showing the read buttons and use appropriate logic gates to ensure that your design exhibits the following:

- Flashes the green LED for valid votes
- Turns ON the red LED for invalid votes
- Prints the vote status to Serial Monitor

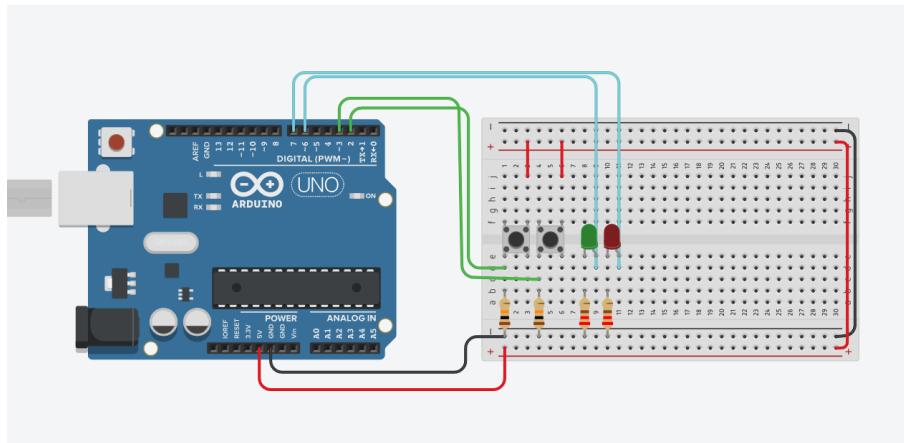


Figure 3.1: Diagram

```

// C++ code
//
class Button {
    private:
        byte pin;
        bool lastState;
        bool debounceState;
        unsigned long lastDebounce;
        void (*riseEdge)() = nullptr;
        void (*fallEdge)() = nullptr;

    public:
        Button(byte p){
            pin=p;
        }
        void begin(){
            pinMode(pin,INPUT);
            lastState = digitalRead(pin);
            debounceState = lastState;
            lastDebounce = millis();
        }
        void setRiseCallback(void (*callback)()) {
            riseEdge = callback;
        }

        void setFallCallback(void (*callback)()) {
            fallEdge = callback;
        }
        void update(){
            bool curState= digitalRead(pin);
            if (lastState!=curState){
                lastDebounce = millis();
            }
            if ((millis() - lastDebounce) > 50){
                if (curState != debounceState) {
                    debounceState = curState;
                    if (debounceState && riseEdge){
                        riseEdge();
                    }
                }
            }
        }
    }

```

```

        else if (!debounceState && fallEdge){
            fallEdge();
        }
    }
    lastState=curState;
}

bool getDebounceState(){
    return debounceState;
}

};

Button buttonA(2);
Button buttonB(3);
const byte greenLED=6;
const byte redLED=7;

bool sigA=false;
bool sigB=false;
bool clash=false;

int Acount=0;
int Bcount=0;

bool XOR(bool sig1,bool sig2){
    return sig1^sig2;
}

bool AND(bool sig1,bool sig2){
    return sig1==sig2;
}

void A_rise(){
    sigA=true;
}

void B_rise(){
    sigB=true;
}

void A_fall(){
    if (clash){
        clash=false;
        sigA=false;
    }
}

```

```

    sigB=false;
    return;
}
clash=AND(sigA,sigB);
if (clash){
    digitalWrite(redLED,HIGH);
    Serial.println("Invalid Vote");
    return;
}
if (XOR(sigA,sigB)){
    Acount++;
    Serial.println("Valid Vote");
    Serial.print("Candidate A: ");
    Serial.print(Acount);
    Serial.print(" Candidate B: ");
    Serial.print(Bcount);
    Serial.println();
    digitalWrite(redLED,LOW);
    sigA=false;
    digitalWrite(greenLED,HIGH);
    delay(500);
    digitalWrite(greenLED,LOW);
}
}
void B_fall(){
    if (clash){
        clash=false;
        sigA=false;
        sigB=false;
        return;
    }
    clash=AND(sigA,sigB);
    if (clash){
        digitalWrite(redLED,HIGH);
        Serial.println("Invalid Vote");
        return;
    }
    if (XOR(sigA,sigB)){
        Bcount++;

```

```

        Serial.println("Valid Vote");
        Serial.print("Candidate A: ");
        Serial.print(Acount);
        Serial.print(" Candidate B: ");
        Serial.print(Bcount);
        Serial.println();
        digitalWrite(redLED,LOW);
        sigB=false;
        digitalWrite(greenLED,HIGH);
        delay(500);
        digitalWrite(greenLED,LOW);
    }
}

void setup()
{
    Serial.begin(9600);
    buttonA.setRiseCallback(A_rise);
    buttonB.setRiseCallback(B_rise);
    buttonA.setFallCallback(A_fall);
    buttonB.setFallCallback(B_fall);
    buttonA.begin();
    buttonB.begin();
    pinMode(greenLED, OUTPUT);
    pinMode(redLED, OUTPUT);
}

void loop()
{
    buttonA.update();
    buttonB.update();
}

```

3.3 Question 3.3

Draw the logic circuit showing how the XOR function can be created using only AND, OR, and NOT gates.

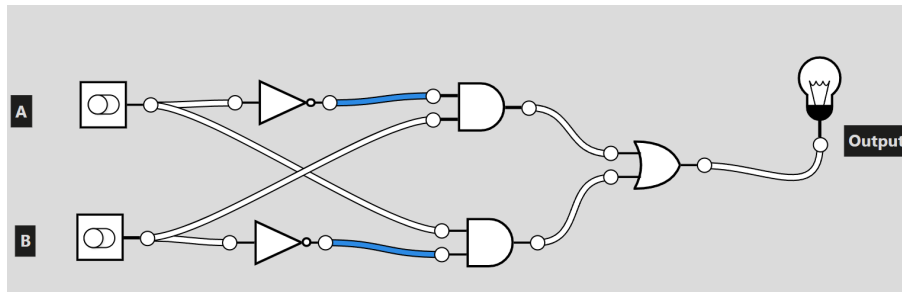


Figure 3.2: XOR Gate Logic Circuit

3.4 Question 3.4

If the current voting number is 91 and your internal memory only stores number either in binary or Hexadecimal. What will be the equivalent binary number and hexadecimal number stored?

$$(91)_{10} = (64)_{10} + (16)_{10} + (8)_{10} + (2)_{10} + (1)_{10}$$

$$(91)_{10} = (2^6)_{10} + (2^4)_{10} + (2^3)_{10} + (2^1)_{10} + (2^0)_{10}$$

$$(91)_{10} = (1011011)_2$$

$$(91)_{10} = (01011011)_2$$

$$(91)_{10} = (5B)_{16}$$

3.5 Question 3.5

Modify the sketch to count the number of valid votes for each candidate and display them using `Serial.println()`.

```
// C++ code
//
class Button {
    private:
        byte pin;
        bool lastState;
        bool debounceState;
        unsigned long lastDebounce;
        void (*riseEdge)() = nullptr;
        void (*fallEdge)() = nullptr;

    public:
        Button(byte p){
            pin=p;
        }
        void begin(){
            pinMode(pin,INPUT);
            lastState = digitalRead(pin);
            debounceState = lastState;
            lastDebounce = millis();
        }
        void setRiseCallback(void (*callback)()) {
            riseEdge = callback;
        }

        void setFallCallback(void (*callback)()) {
            fallEdge = callback;
        }
        void update(){
            bool curState= digitalRead(pin);
            if (lastState!=curState){
                lastDebounce = millis();
            }
        }
    };
};
```

```

    if ((millis() - lastDebounce) > 50){
        if (curState != debounceState) {
            debounceState = curState;
            if (debounceState && riseEdge){
                riseEdge();
            }
            else if (!debounceState && fallEdge){
                fallEdge();
            }
        }
        lastState=curState;
    }
    bool getDebounceState(){
        return debounceState;
    }
};

Button buttonA(2);
Button buttonB(3);
const byte greenLED=6;
const byte redLED=7;

bool sigA=false;
bool sigB=false;
bool clash=false;

int Acount=0;
int Bcount=0;

bool XOR(bool sig1,bool sig2){
    return sig1^sig2;
}

bool AND(bool sig1,bool sig2){
    return sig1==sig2;
}

void A_rise(){
    sigA=true;
}

void B_rise(){

```

```

        sigB=true;
    }
    void A_fall(){
        if (clash){
            clash=false;
            sigA=false;
            sigB=false;
            return;
        }
        clash=AND(sigA,sigB);
        if (clash){
            digitalWrite(redLED,HIGH);
            Serial.println("Invalid Vote");
            return;
        }
        if (XOR(sigA,sigB)){
            Acount++;
            Serial.println("Valid Vote");
            Serial.print("Candidate A: ");
            Serial.print(Acount);
            Serial.print(" Candidate B: ");
            Serial.print(Bcount);
            Serial.println();
            digitalWrite(redLED,LOW);
            sigA=false;
            digitalWrite(greenLED,HIGH);
            delay(500);
            digitalWrite(greenLED,LOW);
        }
    }
    void B_fall(){
        if (clash){
            clash=false;
            sigA=false;
            sigB=false;
            return;
        }
        clash=AND(sigA,sigB);
        if (clash){

```

```

        digitalWrite(redLED,HIGH);
        Serial.println("Invalid Vote");
        return;
    }
    if (XOR(sigA,sigB)){
        Bcount++;
        Serial.println("Valid Vote");
        Serial.print("Candidate A: ");
        Serial.print(Acount);
        Serial.print(" Candidate B: ");
        Serial.print(Bcount);
        Serial.println();
        digitalWrite(redLED,LOW);
        sigB=false;
        digitalWrite(greenLED,HIGH);
        delay(500);
        digitalWrite(greenLED,LOW);
    }
}

void setup()
{
    Serial.begin(9600);
    buttonA.setRiseCallback(A_rise);
    buttonB.setRiseCallback(B_rise);
    buttonA.setFallCallback(A_fall);
    buttonB.setFallCallback(B_fall);
    buttonA.begin();
    buttonB.begin();
    pinMode(greenLED, OUTPUT);
    pinMode(redLED, OUTPUT);
}

void loop()
{
    buttonA.update();
    buttonB.update();
}

```

3.6 Question 3.6

Simplify this logic expression: $Y = AB + A'B + AB'$

$$Y = AB + A'B + AB'$$

$$Y = A(B + B') + A'B = B(A + A') + AB'$$

$$Y = A + A'B = B + AB'$$

$$Y + Y = A + A'B + B + AB'$$

$$Y = A(1 + B') + B(A' + 1)$$

$$\boxed{Y = A + B}$$

3.7 Question 3.7

What is/are the benefits of minimizing logic paths in voting machines?

When designing a Voting machine one can implement the logic in one of two ways: 1) implement logic via software (More memory and processor dependent) and 2) implement logic via hardware (More hardware dependent, requires a larger circuit and more components) With either implementation, it is cheaper both financially and time wise to minimize logic paths. A longer Logic path requires more programming, memory and processing power for method 1) and in method 2) it requires a larger circuit. Either way, it is beneficial to minimize logic paths

Chapter 4: Question 4

A student wired a 555 timer in astable mode to continuously flash an LED and sound a buzzer. However, the buzzer only sounds briefly, and the LED does not blink as expected. The output is inconsistent or absent altogether.

4.1 Question 4.1

Identify and explain two possible wiring or component issues in the astable 555 timer circuit that could cause this behavior. Provide a detailed explanation for each issue and suggest a corrective measure to fix the circuit.

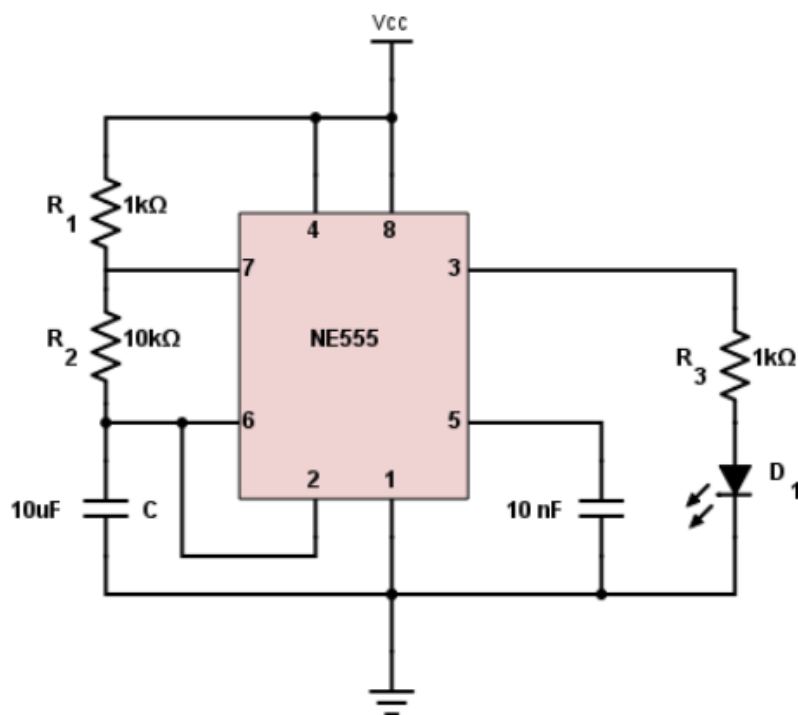


Figure 4.1: 555 Timer in Astable Mode Circuit with a 50% duty cycle and 1Hz frequency

4.1.1 Issue 1

If R_1 and R_2 are swapped, the duty cycle and frequency will change. This results in a duty cycle of 91% and a frequency of 12mHz. In this situation the capacitor will over charge and discharge, meaning the LED will be overly bright and the buzzer overly loud and will then fade to dark and silent before each period is over. This will appear as very unexpected and inconsistent behavior.

4.1.2 Issue 2

If the capacitor is connected to the wrong pin. This will leave pin 6 floating. Additionally, the capacitor will not be charged and discharged meaning no oscillation will occur and the output on pin 3 will be pulled high. This should result in the output doing unexpected things.

4.2 4.2

Simulate the astable behaviour of the 555 timer using an Arduino sketch. The sketch should:

- Continuously toggle a buzzer on pin 8 without using the delay() function.
- Set the ON time of the buzzer to 0.5 seconds and the OFF time to 0.5 seconds (i.e., 1 Hz square wave).
- Run continuously without being affected by button input (no manual trigger).

```
// C++ code
//
const byte buzzer=9;
const byte led=3;
unsigned long timer;
unsigned long curTime;
bool dutyHigh=true;
void setup(){
  pinMode(buzzer, OUTPUT);
  pinMode(led, OUTPUT); // Set buzzer - pin 9 as an output
  timer = millis();
}

void loop(){
  curTime=millis();
  if(curTime-timer>500){
```

```
    if (dutyHigh){
        tone(buzzer, 800);
        digitalWrite(led,HIGH);
        dutyHigh=false;
    }else{
        noTone(buzzer);
        digitalWrite(led,LOW);
        dutyHigh=true;
    }
    timer=curTime;
}
}
```

Bibliography